



**Politechnika Krakowska**

**Wydział Inżynierii Elektrycznej i Komputerowej**

**Katedra Automatyki i Informatyki**

Skrypt do projektu z przedmiotu:

**Grafika Komputerowa**

---

**Zapisywanie i odczytywanie danych**

W ramach czwartego spotkania projektowego spróbujemy zapisać zebrane z obrazu dane do pliku jak również prawidłowo odczytać dane i zwizualizować je na ekranie. Wartości które będziemy chcieli zapisać do pliku domyślnie będą ograniczać się do jednego bajtu, ale postaramy się uwzględnić także przypadki, w których zapisywane jest jednocześnie więcej niż jeden bajt. Aby zweryfikować czy dane zostały zapisane prawidłowo posłużymy się dodatkowym, darmowym, dołączonym do projektu narzędziem `lister.exe` pozwalającym na podgląd plików binarnych. Warto tutaj zwrócić wagę na fakt, iż dane będziemy zapisywać w sposób binarny, tj. np. wartość 250 zostanie zapisana do pliku nie jako trzy znaki '250' tylko jako pojedyncza wartość heksadecymalna FAh.

W C++ istnieje bardzo wiele sposobów na przeprowadzenie operacji na plikach, najłatwiejsza, to kwestia skorzystania z przeciążonego operatora << przystosowanego do pracy ze strumieniami (obiektami typu `ostream`). Spróbujemy w naszym pliku źródłowym `GK2026-Pliki.h` zadeklarować metodę `zapiszPlikv0()` oraz napiszmy jej implementację w pliku `GK2026-Pliki.cpp`:

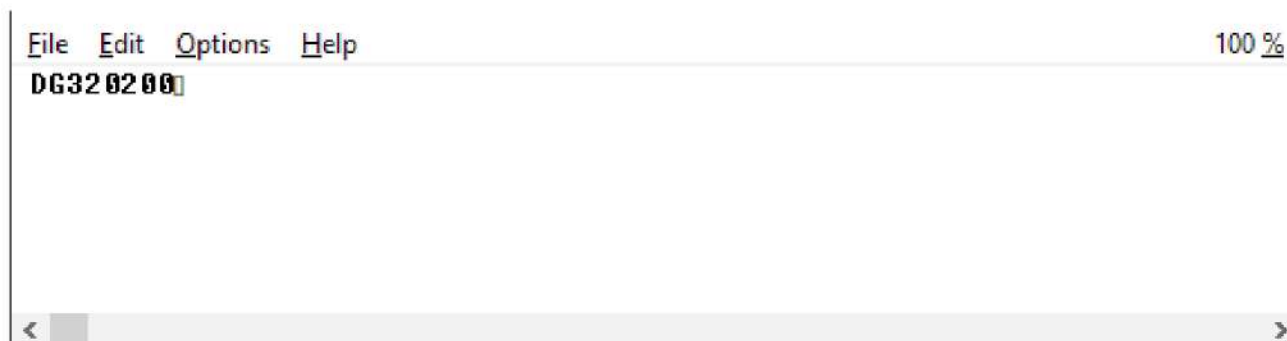
```
void zapiszPlikv0() {  
  
    SDL_Color kolor;  
    Uint16 szerokoscObrazka = szerokosc/2;  
    Uint16 wysokoscObrazka = wysokosc/2;  
    Uint8 ileBitow = 24;  
  
    cout<<"Zapisujemy plik 'obraz.bin' uzywajac operatora <<"<<endl;  
    ofstream wyjście("obraz.bin");  
  
    wyjście<<"DG";  
    wyjście<<szereokoscObrazka;  
    wyjście<<wysokoscObrazka;  
    wyjście<<ileBitow;  
  
    wyjście.close();  
  
    SDL_UpdateWindowSurface(window);  
}
```

W powyższej implementacji deklarujemy cztery zmienne różnych typów (`kolor` typu `SDL_Color`, `szereokoscObrazka` i `wysokoscObrazka` typu `Uint16` oraz `ileBitow` typu `Uint8`) celem ich późniejszego zapisania do pliku. Korzystając z `ofstream` tworzymy plik 'obraz.bin' i przypisujemy go do zmiennej `wyjście`. Następnie przesyłamy do zmiennej `wyjście` zmienne `szereokoscObrazka`, `wysokoscObrazka`, `ileBitow` oraz dodatkowo, jako pierwsze, obiekt typu `String` zawierający ciąg znaków 'DG'.

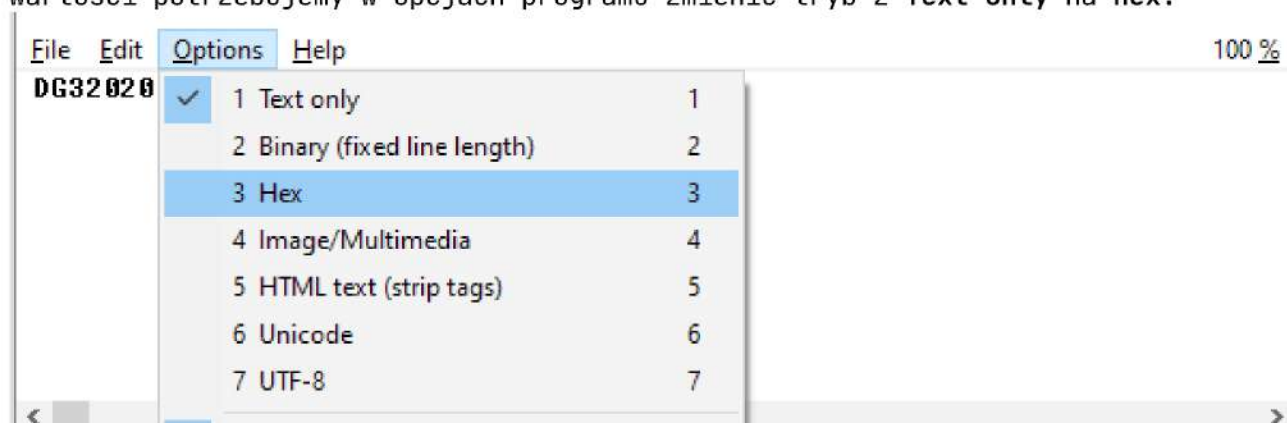
Spróbujemy dopisać do metody `Funkcja9()` wywołanie metody `zapiszPlikv0()` i sprawdzimy jej działanie:

```
void Funkcja9() {  
    zapiszPlikv0();  
    SDL_UpdateWindowSurface(window);  
}
```

Po wywołaniu powyższej funkcji powinien zostać utworzony plik `obraz.bin` zawierający wszystkie zapisane do zmiennej `wyjście` wartości. Otwórzmy z systemu plików narzędzie `lister.exe` i załadujmy plik `obraz.bin`, co powinno doprowadzić do sytuacji w której będziemy widzieć podgląd tego pliku:



Już na tym etapie możemy zaobserwować, że część danych zapisała się zgodnie z naszymi oczekiwaniami ('DG' jest zapisane prawidłowo), ale niestety nie wszystkie zmienne zapisały się w oczekiwanej formie (`szerokoscObrazka` i `wysokoscObrazka` są w formie tekstowej zamiast binarnej). Aby zobaczyć rzeczywiste wartości potrzebujemy w opcjach programu zmienić tryb z **Text only** na **Hex**.



Spowoduje to wyświetlenie zarówno wersji tekstowej (po prawej stronie) jak i heksadecymalnej (po lewej stronie).



Zapisanie naszych 16-bitowych wartości `szerokoscObrazka` i `wysokoscObrazka` w formie tekstowej rzutuje na dwa problemy, tj. wymaga więcej danych niż spodziewane (wartość 16-bitowa powinna zajmować dwa bajty) oraz jest niejednoznaczna (nie jesteśmy w stanie stwierdzić czy `320200` oznacza, że mamy `320` szerokości i `200` wysokości czy może `3` szerokości i `20200` wysokości?). Z kolei możemy zauważyć, że nasza ostatnia zmienna `ileBitow`, zadeklarowana jako wartość 8-bitowa, została zapisana prawidłowo, tj. jako bajt o wartości `18h` (czyli 24 dziesiętnie), a nie jako dwa znaki '24'. Wynika to z faktu, iż operator `<<` obiektu zajmujące więcej niż 8 bitów rzutuje na stringa chcąc przedstawić taką wartość w formie czytelnej dla człowieka (dlatego np. `cout` wyświetlając wartość typu `int` przedstawia ją w czytelnej dla nas formie tekstowej). Z kolei wartość 8-bitowa jest traktowana jako `char` i w takiej formie przesyłana na wyjście (dlatego jeżeli chcieliśmy wyświetlić za pomocą `cout` wartość typu `uint8` musieliśmy rzutować ją na typ `int`).

Skorygujmy działanie naszej metody, aby skoro prawidłowo zapisuje wartości 8-bitowe, otrzymała każdą zmienną w postaci serii wartości 8-bitowych (możemy to zrealizować odpowiednio posługując się rzutowaniem na typ `UInt8` i przesunięciem bitowym):

```
void zapiszPlikv0() {
...
    wyjście<<"DG";
    wyjście<<(UInt8)(szerokoscObrazka)<<(UInt8)(szerokoscObrazka>>8);
    wyjście<<(UInt8)(wysokoscObrazka)<<(UInt8)(wysokoscObrazka>>8);
    wyjście<<ileBitow;
...
}
```

Gdy w tym momencie zerknijemy na zawartość tak utworzonego pliku, to możemy zaobserwować, że wszystkie wartości zostały zapisane w spodziewany sposób:



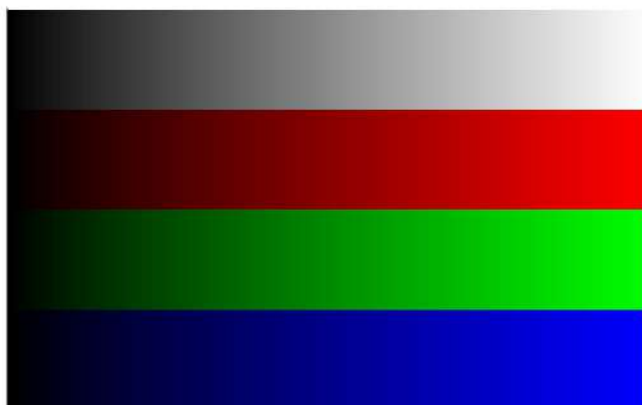
Na powyższym przykładzie możemy zaobserwować, że zmienna `szerokoscObrazu` reprezentowana przez 16-bitową wartość `320` została zapisana jako `0140h` w notacji little endian. Domyślnie stosujemy tutaj wykorzystywaną przez procesory z rodziny x86 notację little endian czyli formę zapisu danych, w której najmniej znaczący bajt, z ang. low-order byte umieszczony jest jako pierwszy, dlatego fizycznie najpierw mamy wartość `40h` (czyli najmniej znaczący bajt), a dopiero później `01h` (czyli najbardziej znaczący bajt). Podobnie ma to miejsce w przypadku zmiennej `wysokoscObrazu` reprezentowanej przez 16-bitową wartość `200`, która została zapisana jako `00C8h` czyli najpierw mamy `C8h`, a później `00h`. Tym razem każda ze zmiennych jest łatwa w zdekodowaniu (16-bitowa wartość to zawsze dwa bajty) i zajmuje w pliku dokładnie tyle ile spodziewamy się, że będzie zajmować.

Dodajmy finalnie do naszego pliku także dane zebrane z obrazu, tj. po informacji o ilości bitów zapiszemy do pliku kombinację RGB składającą się na każdy piksel naszego obrazka:

```
void zapiszPlikv0() {
...
    wyjście<<ileBitow;

    for (int y=0; y<wysokoscObrazka; y++) {
        for (int x=0; x<szerokoscObrazka; x++) {
            kolor = getPixel(x, y);
            wyjście<<kolor.r<<kolor.g<<kolor.b;
        }
    }
    wyjście.close();
    SDL_UpdateWindowSurface(window);
}
```

Testując wspomnianą metodą na ostatnim obrazku



powinniśmy dostać w naszym pliku łatwe do interpretacji wartości będące reprezentacją naszego gradientu – zaraz za informacją o liczbie bitów mamy wartości rozpoczynające się od 00h i systematycznie rosnące:

```
File Edit Options Help 0 %
00000000: 44 47 40 01 C8 00 18 00|00 00 00 00 01 01 01 | pG@č  00000000
00000010: 02 02 02 03 03 03 03|03 04 04 04 05 05 05 | 000000000000
00000020: 06 06 07 07 07 07 07|08 08 08 09 09 09 0A | 000000000000
00000030: 0A 0B 0B 0B 0B 0B 0B|0C 0C 0C 0D 0D 0E 0E | 000000000000
00000040: 0F 0F 0F 0F 0F 10 10|10 11 11 11 12 12 13 | 000000000000
00000050: 13 13 13 13 14 14 14|15 15 15 16 16 16 17 | 000000000000
00000060: 17 17 17 17 18 18 18|19 19 1A 1A 1A 1B 1B | 000000000000
00000070: 1B 1B 1B 1C 1C 1C 1D|1D 1E 1E 1E 1F 1F 1F | 000000000000
00000080: 1F 1F 20 20 20 21 21|22 22 22 23 23 23 23 | 000000000000
00000090: 23 24 24 24 25 25 25|26 26 26 27 27 27 27 | 000000000000
000000A0: 28 28 28 29 29 29 2A|2A 2B 2B 2B 2B 2B 2C | 000000000000
000000B0: 2C 2C 2D 2D 2D 2E 2E|2F 2F 2F 2F 2F 2F 30 | 000000000000
000000C0: 30 31 31 31 32 32 32|33 33 33 33 33 33 34 | 000000000000
000000D0: 35 35 35 36 36 36 36|36 37 37 37 38 38 39 | 000000000000
000000E0: 39 39 3A 3A 3A 3A 3A|3B 3B 3B 3C 3C 3C 3D | 000000000000
000000F0: 3D 3E 3E 3E 3E 3E 3F|3F 3F 40 40 40 41 41 | 000000000000
00000100: 42 42 42 42 42 43 43|43 44 44 44 45 45 46 | 000000000000
00000110: 46 46 46 46 47 47 47|48 48 48 49 49 49 4A | 000000000000
00000120: 4A 4A 4A 4A 4B 4B 4B|4C 4C 4C 4D 4D 4E 4E | 000000000000
00000130: 4E 4E 4E 4F 4F 4F 50|50 51 51 51 52 52 52 | 000000000000
00000140: 52 52 53 53 53 54 54|55 55 55 56 56 56 56 | 000000000000
00000150: 56 57 57 57 58 58 58|59 59 5A 5A 5A 5A 5A | 000000000000
00000160: 5B 5B 5B 5C 5C 5C 5D|5D 5E 5E 5E 5E 5E 5F | 000000000000
00000170: 5F 5F 60 60 60 61 61|62 62 62 62 62 63 63 | 000000000000
00000180: 63 64 64 64 65 65 65|66 66 66 66 66 67 67 | 000000000000
00000190: 68 68 68 69 69 69 69|6A 6A 6A 6B 6B 6B 6C | 000000000000
000001A0: 6C 6C 6D 6D 6D 6D 6D|6E 6E 6E 6F 6F 6F 70 | 000000000000
000001B0: 70 71 71 71 71 71 71|72 72 72 73 73 73 74 | 000000000000
000001C0: 75 75 75 75 75 75 76|76 77 77 77 78 78 79 | 000000000000
000001D0: 79 79 79 79 7A 7A 7A|7B 7B 7B 7C 7C 7C 7D | 000000000000
000001E0: 7D 7D 7D 7D 7E 7E 7E|7F 7F 80 80 80 81 81 | 000000000000
000001F0: 81 81 81 82 82 82 83|83 84 84 84 85 85 85 | 000000000000
```

Metoda zapiszPlikv0() działa prawidłowo, ale nie jest specjalnie wygodna w zastosowaniu (każdorazowa konieczność rzutowania i przesuwania wartości większych niż 8-bitowe stwarza niepotrzebne utrudnienie), dlatego spróbujemy w trochę inny sposób przekazywać dane do naszego pliku. Stworzymy implementację nowej metody zapiszPlik() w której postaramy się zapisać dane w bardziej uniwersalny sposób.

```
void zapiszPlik() {

    SDL_Color kolor;
    Uint16 szerokoscObrazka = szerokosc/2;
    Uint16 wysokoscObrazka = wysokosc/2;
    Uint8 ileBitow = 24;
    char identyfikator[] = "DG";

    cout<<"Zapisujemy plik 'obrazRGB.bin' uzywajac metody write()"<<endl;
    ofstream wyjście("obrazRGB.bin", ios::binary);
    wyjście.write((char*)&identyfikator, sizeof(char)*2);
    wyjście.write((char*)&szerokoscObrazka, sizeof(Uint16));
    wyjście.write((char*)&wysokoscObrazka, sizeof(Uint16));
    wyjście.write((char*)&ileBitow, sizeof(Uint8));

    wyjście.close();

    SDL_UpdateWindowSurface(window);
}
```

W powyższej wersji posługujemy się dedykowaną metodą `write()` pozwalającą na określenie wskaźnika do zmiennej którą chcemy zapisać oraz dodatkowo pozwalającej na określenie rozmiaru zapisywanej zmiennej. Warto także zwrócić uwagę na fakt, że nasz strumień wyjściowy `wyjście` został otwarty z dodatkową flagą `ios::binary` oznaczającą otwarcie pliku w trybie binarnym (w takim trybie znaki nowego wiersza nie mają specjalnego znaczenia i są traktowane tak jak każdy inny bajt). Dodatkowo, dwa znaki zapisywane na początku pliku zostały zadeklarowane jako dwuelementowa tablica typu `char`. Zastąpmy w Funkcji 9 wywołanie starszej wersji funkcji jej nowszym odpowiednikiem i zobaczmy co zostało zapisane do pliku o nazwie `obrazRGB.bin`:

```
void Funkcja9() {
    zapiszPlik();
    SDL_UpdateWindowSurface(window);
}
```



Tym razem od razu mamy w pliku spodziewaną reprezentację poszczególnych zmiennych czyli zapis w którym wartość tekstowa (tablica znaków `identyfikator`) została zapisana w formie tekstowej `'DG'`, wartość 16-bitowa (`szerokoscObrazka` i `wysokoscObrazka`) w postaci dwóch bajtów w notacji little endian `40h 01h` i `C8h 00h`, a wartość 8-bitowa (`ileBitow`) w postaci 1 bajtu `18h`. Dodajmy zatem do implementacji kwestię zbierania danych obrazu i ich zapis do pliku:

```
void zapiszPlik() {
...
    wyjscie.write((char*)&ileBitow, sizeof(UInt8));

    for (int y=0; y<wysokoscObrazka; y++) {
        for (int x=0; x<szerokoscObrazka; x++) {
            kolor = getPixel(x, y);
            wyjscie.write((char*)&kolor.r, sizeof(UInt8));
            wyjscie.write((char*)&kolor.g, sizeof(UInt8));
            wyjscie.write((char*)&kolor.b, sizeof(UInt8));
        }
    }

    wyjscie.close();

    SDL_UpdateWindowSurface(window);
}
```

Jako ciekawostkę warto spróbować nieco przyspieszyć działanie zapisywania danych przesyłając do wyjścia od razu trzy bajty z naszej zmiennej typu `SDL_Color`. Zwróćmy uwagę na fakt, iż sam typ `SDL_Color` zawiera cztery właściwości typu `UInt8` (`r`, `g`, `b` oraz `a`), ale ostatnia z nich, kanał alfa, nie jest przez nas używana, dlatego wymusimy, aby nie cała zmienna typu `SDL_Color`, a jedynie trzy jej pierwsze bajty zostały przekazane do pliku:

```
void zapiszPlik() {
...
    kolor = getPixel(x, y);
    wyjscie.write((char*)&kolor, sizeof(UInt8)*3);
    // wyjscie.write((char*)&kolor.r, sizeof(UInt8));
    // wyjscie.write((char*)&kolor.g, sizeof(UInt8));
    // wyjscie.write((char*)&kolor.b, sizeof(UInt8));
}
    wyjscie.close();

    SDL_UpdateWindowSurface(window);
}
```

Plik utworzony w powyższy sposób jest tożsamy co do wartości jak i rozmiaru z wersją poprzednią. Spróbujmy zatem odczytać tak zgromadzone w pliku dane. Stwórzmy do tego celu metodę `odczytajPlik()` którą wywołamy również `Funkcja9()`:

```
void Funkcja9() {
    zapiszPlik();
    odczytajPlik();
    SDL_UpdateWindowSurface(window);
}
```

Nowa metoda będzie w swojej strukturze mocno zbliżona do wersji zapisującej dane:

```
void odczytajPlik() {
    SDL_Color kolor;
    Uint16 szerokoscObrazka = 0;
    Uint16 wysokoscObrazka = 0;
    Uint8 ileBitow = 0;
    char identyfikator[] = " ";

    cout<<"Odczytujemy plik 'obrazRGB.bin' uzywajac metody read()"<<endl;
    ifstream wejscie("obrazRGB.bin", ios::binary);

    wejscie.read((char*)&identyfikator, sizeof(char)*2);
    wejscie.read((char*)&szerokoscObrazka, sizeof(Uint16));
    wejscie.read((char*)&wysokoscObrazka, sizeof(Uint16));
    wejscie.read((char*)&ileBitow, sizeof(Uint8));

    cout<<"id: "<<identyfikator<<endl;
    cout<<"szerokosc: "<<szerokoscObrazka<<endl;
    cout<<"wysokosc: "<<wysokoscObrazka<<endl;
    cout<<"ile bitow: "<<(int)ileBitow<<endl;

    SDL_UpdateWindowSurface(window);
}
```

W metodzie tej mamy, podobnie jak poprzednio, serię deklaracji zmiennych, ale tym razem zmienne te celowo są zerowane, abyśmy mieli pewność, że wpisane do nich później wartości odczytane z pliku faktycznie z tego pliku pochodzą. Dodatkowo tym razem zamiast strumienia wyjściowego `wyjście` mamy strumień wejściowy `wejscie`. Warto także zwrócić uwagę na fakt, iż do odczytu danych z pliku służy nam metoda `read()` która przyjmuje dokładnie takie same parametry jak miało to miejsce poprzednio przy metodzie `write()`, czyli jest to wskaźnik na zmienną oraz rozmiar odczytywanych danych. Po odczytaniu początkowych danych wyświetlony zostaną na konsoli odczytane wartości. Sprawdźmy czy początkowe dane zostaną prawidłowo odczytane, tj. przetestujmy działanie metody `odczytajPlik()`.

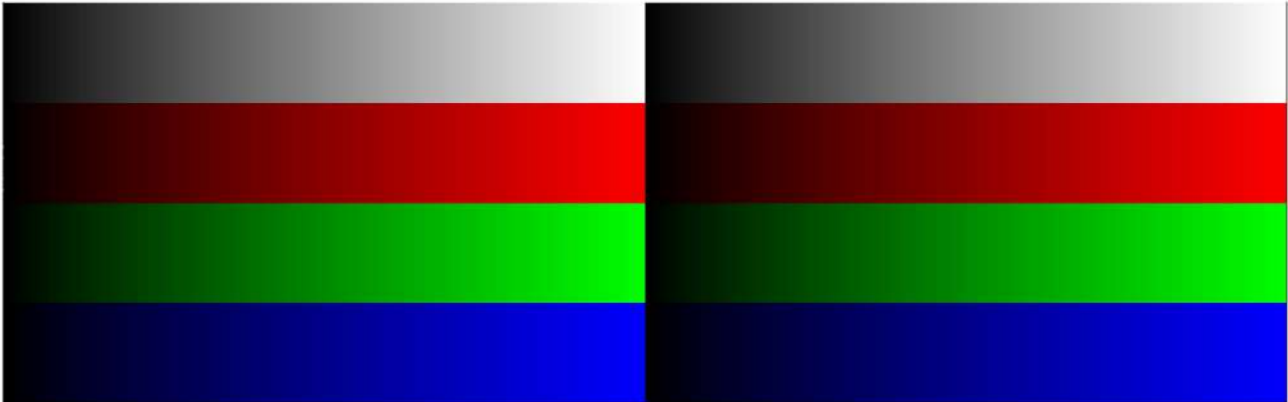
```
Odczytujemy plik 'obrazRGB.bin' uzywajac metody read()
id: DG
szerokosc: 320
wysokosc: 200
ile bitow: 24
```

Jak możemy zauważyć na powyższej kopii informacji zwróconych na konsolę, wskazane dane zostały prawidłowo odczytane. Pozostaje nam zatem uzupełnić metodę o odczyt danych dotyczących obrazu (wartości RGB) oraz ich wizualizację na ekranie:

```
void odczytajPlik() {
    ...
    cout<<"ile bitow: "<<(int)ileBitow<<endl;

    for (int y=0; y<wysokoscObrazka; y++) {
        for (int x=0; x<szerokoscObrazka; x++) {
            wejscie.read((char*)&kolor, sizeof(Uint8)*3);
            setPixel(x+(szerokosc/2), y, kolor.r, kolor.g, kolor.b);
        }
    }
    SDL_UpdateWindowSurface(window);
}
```

Dane dotyczące obrazu odczytane z pliku są wizualizowane bezpośrednio na ekranie ze standardowym przesunięciem o pół ekranu w prawo, co w efekcie powinno nam pozwolić na uzyskanie idealnej kopii naszego oryginalnego obrazu.



Spróbujmy teraz utworzyć dodatkowe metody do zapisywania (`zapiszPlik8()`) i odczytywania (`odczytajPlik8()`) pliku w taki sposób, aby danymi które są zapisywane były nie oryginalne wartości RGB tylko nasz 8-bitowy kolor RGB z jednego z naszych spotkań projektowych.

```
void zapiszPlik8() {
    SDL_Color kolor;
    Uint8 kolor8bit;
    Uint16 szerokoscObrazka = szerokosc/2;
    Uint16 wysokoscObrazka = wysokosc/2;
    Uint8 ileBitow = 8;
    char identyfikator[] = "DG";

    cout<<"Zapisujemy plik 'obraz8.bin' uzywajac metody write()"<<endl;
    ofstream wyjście("obraz8.bin", ios::binary);
    wyjście.write((char*)&identyfikator, sizeof(char)*2);
    wyjście.write((char*)&szerokoscObrazka, sizeof(Uint16));
    wyjście.write((char*)&wysokoscObrazka, sizeof(Uint16));
    wyjście.write((char*)&ileBitow, sizeof(Uint8));

    for (int y=0; y<wysokoscObrazka; y++) {
        for (int x=0; x<szerokoscObrazka; x++) {
            kolor = getPixel(x, y);
            kolor8bit = z24Kdo8K(kolor);
            wyjście.write((char*)&kolor8bit, sizeof(Uint8));
        }
    }

    wyjście.close();

    SDL_UpdateWindowSurface(window);
}
```

Implementacja nowej metody do zapisywania nie jest bardzo skomplikowana, zmieniło się jedynie źródło z którego zapisywane są dane o obrazie, tj. tym razem jest to rezultat działania metody `z24Kdo8K()`. W analogiczny sposób napiszemy metodę do odczytywania wartości 8-bitowych:

```
void odczytajPlik8() {

    SDL_Color kolor;
    Uint8 kolor8bit = 0;
    Uint16 szerokoscObrazka = 0;
    Uint16 wysokoscObrazka = 0;
    Uint8 ileBitow = 0;
    char identyfikator[] = " ";

    cout<<"Odczytujemy plik 'obraz8.bin' uzywajac metody read()"<<endl;

    ifstream wejscie("obraz8.bin", ios::binary);

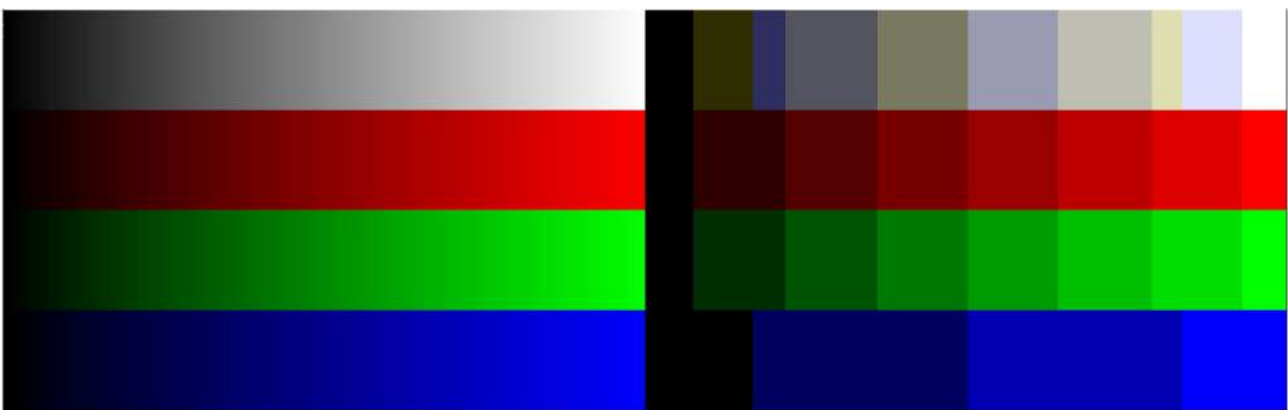
    wejscie.read((char*)&identyfikator, sizeof(char)*2);
    wejscie.read((char*)&szerokoscObrazka, sizeof(Uint16));
    wejscie.read((char*)&wysokoscObrazka, sizeof(Uint16));
    wejscie.read((char*)&ileBitow, sizeof(Uint8));

    cout<<"id: "<<identyfikator<<endl;
    cout<<"szerokosc: "<<szerokoscObrazka<<endl;
    cout<<"wysokosc: "<<wysokoscObrazka<<endl;
    cout<<"ile bitow: "<<(int)ileBitow<<endl;

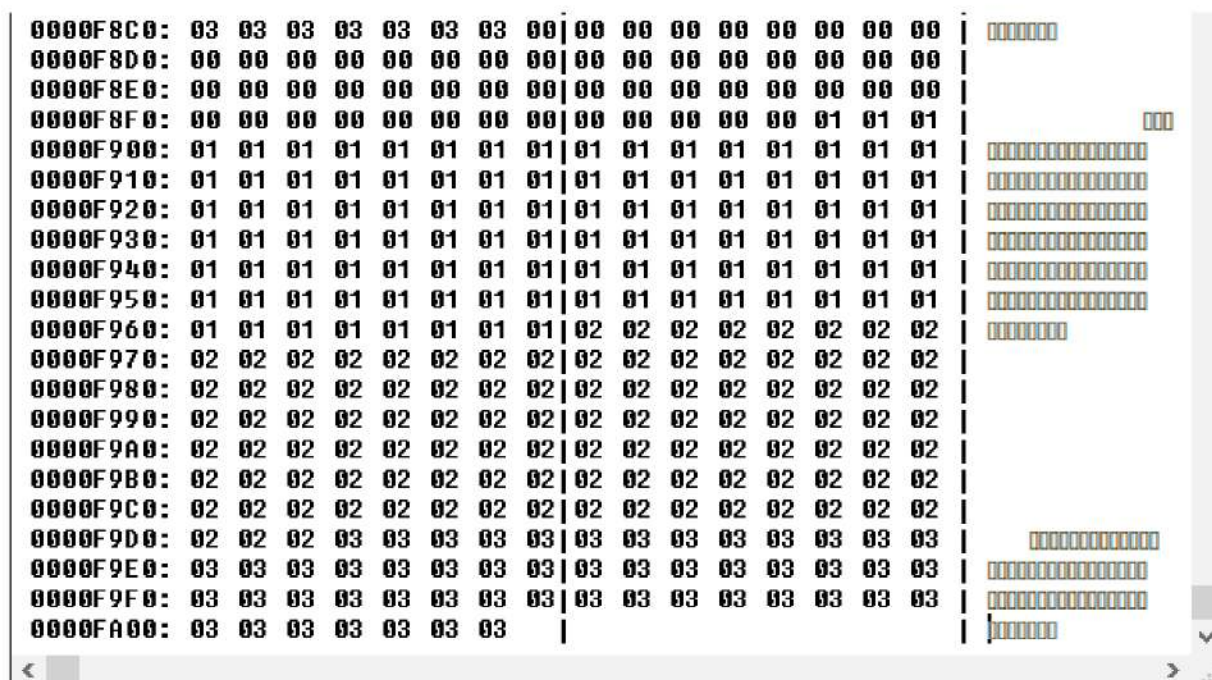
    for (int y=0; y<wysokoscObrazka; y++) {
        for (int x=0; x<szerokoscObrazka; x++) {
            wejscie.read((char*)&kolor8bit, sizeof(Uint8));
            kolor = z8Kdo24K(kolor8bit);
            setPixel(x+(szerokosc/2), y, kolor.r, kolor.g, kolor.b);
        }
    }

    SDL_UpdateWindowSurface(window);
}
```

Wizualizacja działania powyższych metod powinna spowodować wyświetlenie naszego docelowego obrazka zredukowanego do wspomnianego trybu 8-bit RGB.



Warto tutaj zwrócić uwagę na kwestię rozmiaru nowego pliku o nazwie **obraz8.bin**, który to rozmiar jest trzy razy mniejszy niż poprzedniego pliku **obrazRGB.bin**. Wynika to z faktu iż do zapisania informacji o każdym z pikseli potrzebujemy jeden bajt (8-bitów), a nie trzy bajty (24 bity). Jeżeli zerkniemy na zawartość utworzonego pliku, to powinno dać się dość łatwo zinterpretować wartości umieszczonych tam bajtów:



### Zadania do wykonania na zajęciach projektowych:

Za pomocą zaimplementowanych metod mamy możliwość zapisania i odczytania obrazu zarejestrowanego w trybie 8-bit RGB (czyli z tzw. paletą narzuconą). Cechą charakterystyczną tego trybu jest fakt korzystania z wartości z zakresu od 0 do 255, tj. wartości mieszczących się w jednym bajcie. Z analogicznego zakresu wartości korzystamy w przypadku palety dedykowanej stworzonej na podstawie analizy obrazu i wykryciu unikalnych kolorów używanych przez obrazek (z której to palety możemy skorzystać jeżeli kolorów jest nie więcej niż 256), jak również z palety wygenerowanej za pomocą algorytmu MedianCut (ale rozważyć tutaj należy sposób w jaki prześlemy do pliku przygotowaną paletę dedykowaną).

Zadania do wykonania w ramach naszych dzisiejszych zajęć:

1. Stworzenie analogicznych metod pozwalających na zapis i odczyt obrazków n-bitowych, tj. obrazków posługujących się alfabetem wejściowym dla danego zadania projektowego, tj.

Dla wersji 7-bitowej -> 7-bitów, tj. 128 kolorów

Dla wersji 6-bitowej -> 6-bitów, tj. 64 kolory

Dla wersji 5-bitowej -> 5-bitów, tj. 32 kolory

Dla wersji 4-bitowej -> 4-bity, tj. 16 kolorów

Dla wersji 3-bitowej -> 3-bity, tj. 8 kolorów

Istotne jest, aby dane zapisywane były w optymalny sposób, tj. aby np. dla danych 3-bitowych dla każdego z pikseli wykorzystać tylko 3 bity, a nie cały bajt. Możemy to zrealizować odpowiednio przygotowując i optymalizując dane które będą zapisywane do pliku. W przypadku obrazu 3-bitowego optymalizacja może opierać się na tym, że 8 kolejnych wartości 3-bitowych zapiszemy na 3 bajtach kodując odpowiednio poszczególne bity, np. jeżeli nasze kolejne wartości pikseli oznaczmy symbolami **A B C D E F G H**, to:

1 bajt zawiera bity: **A0 B0 C0 D0 E0 F0 G0 H0**

gdzie odpowiednio **A0** to bit nr 0 piksela **A**, **B0** to bit nr 0 piksela **B**, **C0** to bit nr 0 piksela **C**, itd.

2 bajt zawiera bity: **A1 B1 C1 D1 E1 F1 G1 H1**

3 bajt zawiera bity: **A2 B2 C2 D2 E2 F2 G2 H2**,

Taki zapis spowoduje, że dane dotyczące obrazu będą wymagały nie 64000 bajtów ( $320 \times 200 \times 8b = 64000$ ) tylko 24000 ( $320 \times 200 \times 3b$ ).

Dla wersji 4-bitowej poszczególne bajty mogą prezentować się następująco:

1 bajt: **A0 B0 C0 D0 E0 F0 G0 H0**

2 bajt: **A1 B1 C1 D1 E1 F1 G1 H1**

3 bajt: **A2 B2 C2 D2 E2 F2 G2 H2**

4 bajt: **A3 B3 C3 D3 E3 F3 G3 H3**

Dla wersji 5-bitowej poszczególne bajty mogą prezentować się następująco:

1 bajt: **A0 B0 C0 D0 E0 F0 G0 H0**

2 bajt: **A1 B1 C1 D1 E1 F1 G1 H1**

3 bajt: **A2 B2 C2 D2 E2 F2 G2 H2**

4 bajt: **A3 B3 C3 D3 E3 F3 G3 H3**

5 bajt: **A4 B4 C4 D4 E4 F4 G4 H4**

Dla wersji 6-bitowej poszczególne bajty mogą prezentować się następująco:

1 bajt: A0 B0 C0 D0 E0 F0 G0 H0

2 bajt: A1 B1 C1 D1 E1 F1 G1 H1

3 bajt: A2 B2 C2 D2 E2 F2 G2 H2

4 bajt: A3 B3 C3 D3 E3 F3 G3 H3

5 bajt: A4 B4 C4 D4 E4 F4 G4 H4

6 bajt: A5 B5 C5 D5 E5 F5 G5 H5

Dla wersji 7-bitowej poszczególne bajty mogą prezentować się następująco:

1 bajt: A0 B0 C0 D0 E0 F0 G0 H0

2 bajt: A1 B1 C1 D1 E1 F1 G1 H1

3 bajt: A2 B2 C2 D2 E2 F2 G2 H2

4 bajt: A3 B3 C3 D3 E3 F3 G3 H3

5 bajt: A4 B4 C4 D4 E4 F4 G4 H4

6 bajt: A5 B5 C5 D5 E5 F5 G5 H5

7 bajt: A6 B6 C6 D6 E6 F6 G6 H6

2. Przystosowane metod z zadania 1 do zbierania i odtwarzania danych w niestandardowy sposób. Kolejność zbierania danych jest następująca:

Pierwszy blok danych zawiera 8 pikseli zbieranych od lewej do prawej strony rozpoczynając od współrzędnej (0, 0), są to kolejno piksele o współrzędnych: (0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (5, 0), (6, 0), (7, 0).

Kolejny blok danych zawiera piksele z drugiego wiersza, czyli rozpoczynając od współrzędnej (0, 1), są to kolejno piksele o współrzędnych:

(0, 1), (1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (6, 1), (7, 1).

Kolejny blok danych zawiera piksele z trzeciego wiersza, itd.

W momencie dotarcia do dolnej krawędzi obrazu następny blok danych wraca do pierwszego wiersza obrazu i zawiera piksele rozpoczynające się od współrzędnej (8, 0), są to kolejno piksele o współrzędnych:

(8, 0), (9, 0), (10, 0), (11, 0), (12, 0), (13, 0), (14, 0), (15, 0).

Kolejny blok danych zawiera piksele z drugiego wiersza, czyli rozpoczynając od współrzędnej (8, 1), są to kolejno piksele o współrzędnych:

(8, 1), (9, 1), (10, 1), (11, 1), (12, 1), (13, 1), (14, 1), (15, 1).

Kolejny blok danych zawiera piksele z trzeciego wiersza, itd. Po dotarciu do dolnej krawędzi obrazu kolejny blok danych wraca do pierwszego wiersza obrazu i zawiera piksele z kolejnej 8-pikselowej kolumny, tj. zawiera piksele rozpoczynające się od współrzędnej (16, 0), są to kolejno piksele o współrzędnych:

(16, 0), (17, 0), (18, 0), (19, 0), (20, 0), (21, 0), (22, 0), (23, 0). itd.

Aby lepiej przedstawić specyficzną kolejność zbierania danych, w poniższej tabelce (symbolizującej hipotetyczny obrazek o rozmiarze 24x16) wartości poszczególnych komórek oznaczają kolejny numer zbieranej z obrazu wartości (liczonej od 0). Kolor poszczególnych komórek ma jedynie wizualizować bloki leżących obok siebie 8 pikseli.

|    | 0   | 1   | 2   | 3   | 4   | 5   | 6   | 7   | 8   | 9   | 10  | 11  | 12  | 13  | 14  | 15  | 16  | 17  | 18  | 19  | 20  | 21  | 22  | 23  |
|----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 0  | 0   | 1   | 2   | 3   | 4   | 5   | 6   | 7   | 128 | 129 | 130 | 131 | 132 | 133 | 134 | 135 | 256 | 257 | 258 | 259 | 260 | 261 | 262 | 263 |
| 1  | 8   | 9   | 10  | 11  | 12  | 13  | 14  | 15  | 136 | 137 | 138 | 139 | 140 | 141 | 142 | 143 | 264 | 265 | 266 | 267 | 268 | 269 | 270 | 271 |
| 2  | 16  | 17  | 18  | 19  | 20  | 21  | 22  | 23  | 144 | 145 | 146 | 147 | 148 | 149 | 150 | 151 | 272 | 273 | 274 | 275 | 276 | 277 | 278 | 279 |
| 3  | 24  | 25  | 26  | 27  | 28  | 29  | 30  | 31  | 152 | 153 | 154 | 155 | 156 | 157 | 158 | 159 | 280 | 281 | 282 | 283 | 284 | 285 | 286 | 287 |
| 4  | 32  | 33  | 34  | 35  | 36  | 37  | 38  | 39  | 160 | 161 | 162 | 163 | 164 | 165 | 166 | 167 | 288 | 289 | 290 | 291 | 292 | 293 | 294 | 295 |
| 5  | 40  | 41  | 42  | 43  | 44  | 45  | 46  | 47  | 168 | 169 | 170 | 171 | 172 | 173 | 174 | 175 | 296 | 297 | 298 | 299 | 300 | 301 | 302 | 303 |
| 6  | 48  | 49  | 50  | 51  | 52  | 53  | 54  | 55  | 176 | 177 | 178 | 179 | 180 | 181 | 182 | 183 | 304 | 305 | 306 | 307 | 308 | 309 | 310 | 311 |
| 7  | 56  | 57  | 58  | 59  | 60  | 61  | 62  | 63  | 184 | 185 | 186 | 187 | 188 | 189 | 190 | 191 | 312 | 313 | 314 | 315 | 316 | 317 | 318 | 319 |
| 8  | 64  | 65  | 66  | 67  | 68  | 69  | 70  | 71  | 192 | 193 | 194 | 195 | 196 | 197 | 198 | 199 | 320 | 321 | 322 | 323 | 324 | 325 | 326 | 327 |
| 9  | 72  | 73  | 74  | 75  | 76  | 77  | 78  | 79  | 200 | 201 | 202 | 203 | 204 | 205 | 206 | 207 | 328 | 329 | 330 | 331 | 332 | 333 | 334 | 335 |
| 10 | 80  | 81  | 82  | 83  | 84  | 85  | 86  | 87  | 208 | 209 | 210 | 211 | 212 | 213 | 214 | 215 | 336 | 337 | 338 | 339 | 340 | 341 | 342 | 343 |
| 11 | 88  | 89  | 90  | 91  | 92  | 93  | 94  | 95  | 216 | 217 | 218 | 219 | 220 | 221 | 222 | 223 | 344 | 345 | 346 | 347 | 348 | 349 | 350 | 351 |
| 12 | 96  | 97  | 98  | 99  | 100 | 101 | 102 | 103 | 224 | 225 | 226 | 227 | 228 | 229 | 230 | 231 | 352 | 353 | 354 | 355 | 356 | 357 | 358 | 359 |
| 13 | 104 | 105 | 106 | 107 | 108 | 109 | 110 | 111 | 232 | 233 | 234 | 235 | 236 | 237 | 238 | 239 | 360 | 361 | 362 | 363 | 364 | 365 | 366 | 367 |
| 14 | 112 | 113 | 114 | 115 | 116 | 117 | 118 | 119 | 240 | 241 | 242 | 243 | 244 | 245 | 246 | 247 | 368 | 369 | 370 | 371 | 372 | 373 | 374 | 375 |
| 15 | 120 | 121 | 122 | 123 | 124 | 125 | 126 | 127 | 248 | 249 | 250 | 251 | 252 | 253 | 254 | 255 | 376 | 377 | 378 | 379 | 380 | 381 | 382 | 383 |